DESENVOLVIMENTO MOBILE · MÓDULO 08 · TRILHA KOTLIN · NÍVEL INTERMEDIÁRIO

Arquitetura de Apps

Camadas, MVVM, UiState, repositórios, casos de uso, injeção de dependência com Hilt e organização do projeto

Prof. Dr. Francisco dos Santos Viana

São Luís – Maranhão – Brasil · 2026

Objetivos do módulo

Ao final deste módulo você será capaz de

- Explicar os princípios da arquitetura recomendada para apps Android.
- Separar o app em camadas de UI, domínio e dados.
- Implementar o padrão MVVM com `ViewModel`, `UiState` e eventos.
- Criar repositórios como fonte única da verdade.
- Encapsular regras de negócio em casos de uso.
- Configurar injeção de dependência com Hilt e organizar pacotes e módulos.

Roteiro

Neste módulo

- Fundamentos: arquitetura de software
- Por que arquitetura?
- Camada de UI: MVVM
- Camada de dados
- Camada de domínio
- Injeção de dependência com Hilt
- Organização do projeto
- Fechamento

Fundamentos: arquitetura de software

O que é arquitetura de software

Arquitetura

As grandes estruturas de um sistema: seus elementos, as relações entre eles e as decisões **difíceis de mudar depois**. Ela existe para proteger atributos de qualidade.

Atributo	Pergunta no LabMonitor
Manutenibilidade	trocar a API exige mexer nas telas?
Testabilidade	dá para testar a regra de reserva sem um aparelho?
Desempenho	a lista de leituras rola sem engasgar?
Funcionamento *offline*	o app mostra dados sem internet?
Segurança	o *token* fica protegido?

Analogia

A planta de uma casa decide por onde passam os canos antes da primeira parede. Pintar um quarto não é arquitetura; mudar a cozinha de lugar é.

Boa arquitetura em um app

Permite mudar a interface sem tocar nos dados, trocar a API sem tocar na interface e testar regras sem abrir o app.

Dica

Coesão, acoplamento, SOLID e padrões de projeto foram vistos no Módulo 03: aqui eles ganham escala.

Padrões de apresentação: de MVC a MVI

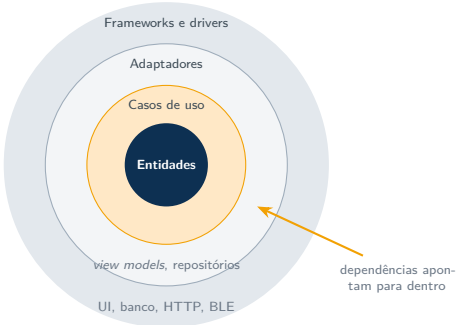


Padrão	Quem atualiza a tela	Ponto forte	Ponto fraco
MVC	o *controller* manipula a *view*	simples	no Android, a `Activity` vira um *controller* gigante
MVP	o *presenter* chama métodos de uma interface da *view*	testável sem a tela	muito código de interface; guarda referência à *view*
MVVM	a *view* observa o estado exposto pelo *view model*	casa com UI declarativa e sobrevive à rotação	vários fluxos de estado podem se desencontrar
MVI	estado único e imutável; cada intenção gera um novo estado	previsível e fácil de depurar	cerimônia demais para telas simples

Na prática atual

Compose e Flutter usam MVVM com um estado único por tela (`UiState`): uma mistura de MVVM com a disciplina do MVI.

Arquitetura limpa e arquitetura hexagonal



Regra da dependência

O código das camadas internas não conhece as externas: as regras de negócio não importam nada de Android, Flutter, HTTP ou SQL.

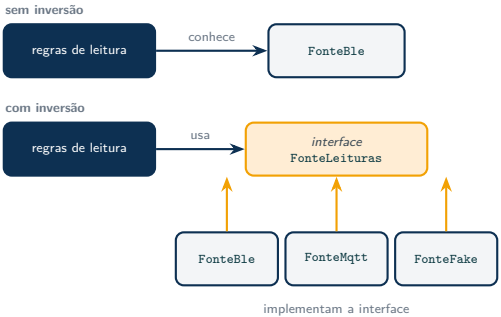
Um pouco de história

Alistair Cockburn propôs a arquitetura hexagonal, de portas e adaptadores, em 2005. Robert C. Martin reuniu ideias parecidas na Arquitetura Limpa, em 2012.

Na medida certa

Em apps pequenos, camadas de UI e de dados bastam. Crie a camada de domínio quando regras se repetirem ou precisarem de testes isolados, como recomendam o Android e o Flutter.

Inversão de dependência na prática



Dart

```dart
abstract interface class FonteLeituras {
  Stream<Leitura> leituras();
}
```

Inversão de dependência

Módulos de alto nível não dependem de detalhes; ambos dependem de abstrações. Quem precisa define a interface; a camada de dados a implementa.

Kotlin

```kotlin
interface FonteLeituras {
  fun leituras(): Flow<Leitura>
}
```

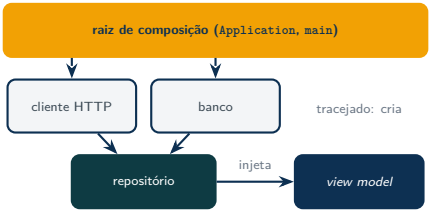
O que se ganha

Trocar BLE por MQTT sem mexer nas regras, testar com uma fonte falsa e fazer a tela antes de o hardware existir.

Injeção de dependência e localizador de serviços

Inversão de controle

A classe não cria o que usa: alguém de fora monta os objetos e os entrega. É o princípio de Hollywood: "não nos chame, nós chamamos você".



Um pouco de história

Martin Fowler batizou o padrão de injeção de dependência em 2004.

Abordagem	Como funciona
Construtor	recebe tudo pronto; dependências explícitas e fáceis de trocar em testes
Contêiner	uma biblioteca monta o grafo: Hilt, Koin; Provider, Riverpod
Localizador	a classe pede a um registro global (`get_it`): prático, mas esconde dependências e só falha em execução

Tempo de vida (escopo)

Único no app: cliente HTTP, banco. **Por tela**: *view model*. **Por uso**: objeto novo a cada pedido. Escopo errado causa vazamento ou estado compartilhado por engano.

Decisões de arquitetura: trocas e registros

Toda decisão é uma troca

Mais camadas trazem isolamento e testabilidade, mas também mais arquivos e indireção. Pese o tamanho do time, a vida esperada do app e o custo de errar.

Princípio	Significado
YAGNI	não construa o que ainda não é necessário
KISS	prefira a solução mais simples que funciona
DRY	cada conhecimento em um só lugar
Regra de três	abstraia na terceira repetição, não na primeira

ADR (*Architecture Decision Record*)

Texto curto, versionado com o código, que registra uma decisão: contexto, opções, decisão e consequências. Responde no futuro: "por que fizemos assim?".

ADR 007 — Estado das telas

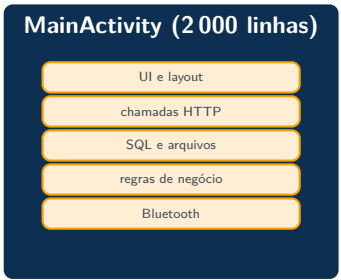
Contexto: leituras do sensor em tempo real. **Decisão**: um `UiState` imutável por tela, exposto como fluxo observável. **Consequência**: telas testáveis sem interface.

Um pouco de história

Michael Nygard propôs os ADRs em 2011.

Por que arquitetura?

O problema da “Activity faz-tudo”



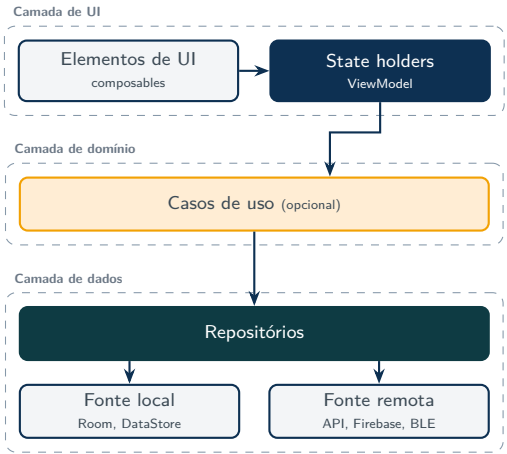
Consequências

- Rotação de tela refaz requisições e perde dados.
- Impossível testar regras sem abrir o app.
- Uma mudança na API quebra a tela inteira.

Princípios recomendados pelo Google

- Separação de responsabilidades.
- UI guiada por modelos de dados.
- Fonte única da verdade.
- Fluxo unidirecional de dados (UDF).

A arquitetura em camadas



- As setas indicam **dependência**: cada camada só conhece a de baixo.
- A camada de dados **não sabe** que existe UI.
- Os dados sobem como `Flow`; os eventos descem como chamadas de função.

Dica

Comece com UI + dados. Crie a camada de domínio quando regras se repetirem em mais de um ViewModel.

Camada de UI: MVVM

MVVM e fluxo unidirecional



View

Desenha o `UiState` e repassa eventos. Não decide nada.

ViewModel

Sobrevive à rotação, transforma dados em `UiState` e trata eventos.

Model

Dados e regras do app, sem nenhuma dependência de Android UI.

Atenção

O ViewModel **nunca** guarda referência a `Activity`, `Context` de tela ou composables: ele vive mais que eles e causaria vazamento de memória.

Modelando o `UiState`

Estado como data class

```
data class PainelUiState(
  val carregando: Boolean = true,
  val bancadas: List<BancadaUi> = emptyList(),
  val filtro: String = "",
  val mensagem: String? = null    // ex.: Snackbar
) {
  val vazio get() = !carregando && bancadas.isEmpty()
}

data class BancadaUi(
  val id: Int,
  val titulo: String,
  val temperatura: String,       // já formatada
  val emAlerta: Boolean
)
```

Ou como sealed interface

```
sealed interface DetalheUiState {
  data object Carregando : DetalheUiState
  data class Pronto(val eq: Equipamento)
    : DetalheUiState
  data class Erro(val msg: String)
    : DetalheUiState
}
```

Qual escolher?

sealed: estados exclusivos (carregando, pronto ou erro). **data class**: aspectos que convivem (lista, filtro, mensagem).

Conceito

O `UiState` é **imutável**: cada mudança gera uma cópia com `copy()`.

O ViewModel

PainelViewModel.kt

```
class PainelViewModel(
  private val repo: BancadaRepository
) : ViewModel() {
  private val filtro = MutableStateFlow("")
  val uiState: StateFlow<PainelUiState> =
    combine(repo.bancadas(), filtro) { lista, f ->
      PainelUiState(
        carregando = false,
        bancadas = lista
          .filter { f.isBlank() || f in it.nome }
          .map { it.paraUi() },
        filtro = f
      )
    }.stateIn(viewModelScope,
      SharingStarted.WhileSubscribed(5_000),
      PainelUiState())

  fun onFiltroChange(novo: String) { filtro.value = novo }
  fun onAtualizar() =
    viewModelScope.launch { repo.sincronizar() }
}
```

- Entradas: fluxos do repositório e estado local (`filtro`).
- `combine` gera um novo `UiState` sempre que algo muda.
- `stateIn` entrega um `StateFlow` pronto para a UI.
- Eventos são funções públicas com nomes de intenção (`onAtualizar`).

Dica

`paraUi()` é uma função de mapeamento: formata números e datas **fora** da UI, o que facilita testar.

Tela: *route* + conteúdo sem estado

TelaPainel.kt

```kotlin
@Composable  // liga o ViewModel à tela (sem lógica)
fun PainelRoute(
  onAbrir: (Int) -> Unit,
  vm: PainelViewModel = hiltViewModel()
) {
  val estado by vm.uiState.collectAsStateWithLifecycle()
  TelaPainel(
    estado = estado,
    onFiltroChange = vm::onFiltroChange,
    onAtualizar = vm::onAtualizar,
    onAbrir = onAbrir
  )
}
@Composable  // só desenha: fácil de testar
fun TelaPainel(
  estado: PainelUiState,
  onFiltroChange: (String) -> Unit,
  onAtualizar: () -> Unit,
  onAbrir: (Int) -> Unit
) { /* Scaffold, campo de busca, LazyColumn... */ }
```

Separação Route / Screen

Route: obtém o ViewModel e coleta o estado.
Screen: recebe estado e lambdas; não conhece o ViewModel.

Preview com qualquer estado

`TelaPainel(PainelUiState(carregando = false, bancadas = exemplo), {}, {}, {})`

Pré-visualize vazio, carregando, erro e lista cheia sem rodar o app.

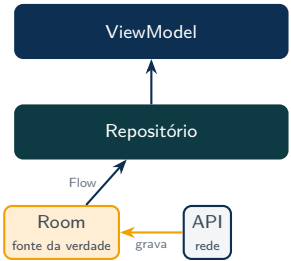
Camada de dados

Repositório: fonte única da verdade

BancadaRepository.kt

```kotlin
interface BancadaRepository {
  fun bancadas(): Flow<List<Bancada>>
  suspend fun sincronizar()
  suspend fun reservar(id: Int, ate: Instant)
}

class BancadaRepositoryImpl(
  private val dao: BancadaDao,        // local (M10)
  private val api: LabApi             // remota (M11)
) : BancadaRepository {
  // a UI sempre lê do banco local
  override fun bancadas() = dao.observarTodas()
    .map { lista -> lista.map { it.paraModelo() } }
  // a rede só atualiza o banco
  override suspend fun sincronizar() {
    val remotas = api.listarBancadas()
    dao.substituirTodas(remotas.map { it.paraEntidade() })
  }
  override suspend fun reservar(id: Int, ate: Instant) =
    api.reservar(id, ate)
}
```



Por que interface?

A UI depende da **abstração**. Nos testes trocamos a implementação por um *fake* em memória.

Um modelo para cada camada

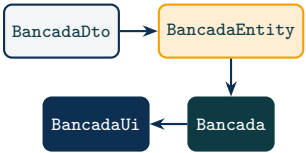
Modelos e mapeamentos

```
// Rede (DTO): espelha o JSON da API
@Serializable
data class BancadaDto(
  val id: Int, val nome: String,
  @SerialName("temp_c") val temp: Double?)

// Banco (Entity): espelha a tabela
@Entity data class BancadaEntity(
  @PrimaryKey val id: Int, val nome: String,
  val temp: Double?)

// Domínio: o que o app entende
data class Bancada(val id: Int, val nome: String,
  val temperatura: Double?)

fun BancadaDto.paraEntidade() =
  BancadaEntity(id, nome, temp)
fun BancadaEntity.paraModelo() =
  Bancada(id, nome, temp)
```



Por que tantos modelos?

Se a API renomear um campo, só o DTO e um mapeamento mudam. O resto do app nem fica sabendo.

Dica

Em apps pequenos é aceitável usar o mesmo modelo em duas camadas. Separe quando os formatos começarem a divergir.

Camada de domínio

Casos de uso

VerificarAlertasUseCase.kt

```
class VerificarAlertasUseCase(
  private val bancadas: BancadaRepository,
  private val ajustes: AjustesRepository
) {
  // operator invoke: o objeto é chamado como função
  operator fun invoke(): Flow<List<Alerta>> =
    combine(bancadas.bancadas(), ajustes.limite()) {
      lista, limite ->
      lista.mapNotNull { b ->
        val t = b.temperatura ?: return@mapNotNull null
        if (t > limite) Alerta(b.id, t, limite) else null
      }
    }
}

// no ViewModel
val alertas = verificarAlertas()
  .stateIn(viewModelScope, WhileSubscribed(5_000),
    emptyList())
```

Caso de uso

Classe com **uma única responsabilidade** de negócio, nomeada com verbo: `ReservarBancada`, `CalcularConsumo`, `VerificarAlertas`.

Quando criar

Quando a regra é usada por vários ViewModels, combina repositórios ou o ViewModel cresceu demais.

Atenção

Domínio é Kotlin puro, sem `Context` nem classes Android, e por isso é reaproveitável em Kotlin Multiplatform.

Injeção de dependência com Hilt

O que é injeção de dependência

Sem DI: a classe cria o que usa

```
class PainelViewModel : ViewModel() {
  private val repo = BancadaRepositoryImpl(
    AppDatabase.get(???).bancadaDao(),
    Retrofit.Builder()...create(LabApi::class)
  )  // acoplado e impossível de testar
}
```

Com DI: recebe pronto

```
class PainelViewModel(
  private val repo: BancadaRepository
) : ViewModel()
```

Injeção de dependência (DI)

Uma classe **recebe** suas dependências (pelo construtor) em vez de criá-las. Alguém de fora monta o "grafo" de objetos.

- **Teste**: injete um repositório *fake*.
- **Reuso**: um único banco e um único cliente HTTP para o app todo.
- **Troca**: mude a implementação em um só lugar.

Opções

Hilt (oficial Google, verificado em compilação), **Koin** (simples, também multiplataforma) ou DI manual com uma classe `AppContainer`.

Hilt: configuração

app/build.gradle.kts

```
plugins {
  alias(libs.plugins.ksp)
  alias(libs.plugins.hilt)
}
dependencies {
  implementation(libs.hilt.android)
  ksp(libs.hilt.compiler)
  implementation(libs.hilt.navigation.compose)
}
```

LabMonitorApp.kt

```
@HiltAndroidApp
class LabMonitorApp : Application()
```

MainActivity.kt

```
@AndroidEntryPoint
class MainActivity : ComponentActivity() { /*...*/ }
```

1. Adicione os plugins e as dependências.
2. Crie uma classe `Application` anotada com `@HiltAndroidApp` e registre-a no manifesto (`android:name=".LabMonitorApp"`).
3. Anote a Activity com `@AndroidEntryPoint`.
4. Anote ViewModels com `@HiltViewModel`.
5. Ensine o Hilt a criar interfaces e classes de bibliotecas com `@Module`.

Dica

O **KSP** (*Kotlin Symbol Processing*) gera o código de DI em tempo de compilação: erros de configuração aparecem no build, não no celular.

Hilt: ViewModels e módulos

ViewModel injetado

```
@HiltViewModel
class PainelViewModel @Inject constructor(
  private val repo: BancadaRepository,
  private val verificarAlertas: VerificarAlertasUseCase
) : ViewModel() { /* ... */ }

// Classes próprias: basta @Inject no construtor
class VerificarAlertasUseCase @Inject constructor(
  private val bancadas: BancadaRepository,
  private val ajustes: AjustesRepository
)

// Na tela
val vm: PainelViewModel = hiltViewModel()
```

di/DataModule.kt

```
@Module
@InstallIn(SingletonComponent::class)
abstract class RepositoryModule {
  @Binds @Singleton   // interface -> implementação
  abstract fun bancadas(
    impl: BancadaRepositoryImpl
  ): BancadaRepository
}

@Module
@InstallIn(SingletonComponent::class)
object DatabaseModule {
  @Provides @Singleton  // classes de biblioteca
  fun banco(@ApplicationContext ctx: Context) =
    Room.databaseBuilder(ctx, AppDatabase::class.java,
      "labmonitor.db").build()

  @Provides fun dao(db: AppDatabase) = db.bancadaDao()
}
```

Organização do projeto

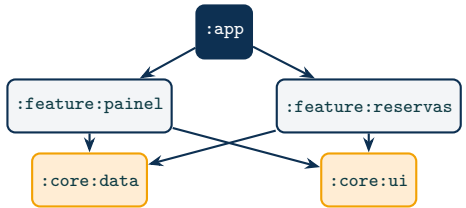
Pacotes por funcionalidade e modularização

Pacotes por feature

```
br.edu.ifma.labmonitor/
|-- LabMonitorApp.kt
|-- MainActivity.kt
|-- navegacao/
|-- core/
|   |-- data/   (db, rede, repositórios)
|   |-- domain/ (modelos, casos de uso)
|   `-- ui/     (tema, componentes comuns)
|-- feature/
|   |-- painel/
|   |   |-- PainelViewModel.kt
|   |   `-- TelaPainel.kt
|   |-- equipamentos/
|   `-- reservas/
`-- di/
```

Pacotes por funcionalidade

Agrupar por *feature* em vez de por tipo mantém juntos os arquivos que mudam juntos.



Modularização Gradle

Em projetos grandes, cada pasta vira um módulo Gradle, com builds mais rápidos e fronteiras garantidas pelo compilador (veja o *Now in Android*).

Fechamento

Resumo

Resumo

- Separe o app em camadas: UI (composables + ViewModel), domínio (casos de uso) e dados (repositórios + fontes).
- MVVM com UDF: o ViewModel expõe `StateFlow<UiState>` e recebe eventos como funções.
- Divida cada tela em *Route* (liga o ViewModel) e *Screen* (só desenha).
- O repositório é a fonte única da verdade; a rede alimenta o banco local.
- DTO, Entity e modelo de domínio isolam o app de mudanças externas.
- Hilt monta o grafo de dependências: `@HiltAndroidApp`, `@HiltViewModel`, `@Inject`, `@Module`.

Glossário (1/2)

Termo	Significado
Arquitetura	estruturas do sistema e decisões difíceis de mudar depois
Camada	grupo de código com uma responsabilidade: UI, domínio ou dados
MVVM	a *view* observa o estado exposto pelo `ViewModel`
MVI	estado único e imutável; cada intenção do usuário gera um novo estado
`UiState`	classe imutável com tudo o que a tela precisa desenhar
`ViewModel`	guarda o estado da tela e sobrevive à rotação
Repositório	fonte única da verdade de um tipo de dado
Caso de uso	classe com uma única regra de negócio reutilizável
Regra da dependência	camadas internas não conhecem as externas

Glossário (2/2)

Termo	Significado
Inversão de dependência	depender de abstrações, não de implementações
Injeção de dependência	receber as dependências prontas em vez de criá-las
Raiz de composição	único lugar onde os objetos concretos são criados e ligados
Localizador de serviços	registro global ao qual a classe pede suas dependências
Hilt	biblioteca de DI do Google, com o grafo verificado na compilação
Escopo	tempo de vida de uma dependência (`@Singleton`, `@ViewModelScoped`)
ADR	registro curto de uma decisão de arquitetura e de seus motivos

Questões de revisão

1. Que problemas aparecem quando uma `Activity` faz tudo? Cite três.
2. Compare MVP e MVVM: quem atualiza a tela em cada um?
3. Por que o `UiState` deve ser imutável e único por tela?
4. Enuncie a regra da dependência e dê um exemplo de violação.
5. Como a interface `FonteLeituras` permite trocar BLE por MQTT sem mexer no `ViewModel`?
6. Qual a diferença entre injeção de dependência e localizador de serviços?
7. Que escopo dar ao cliente HTTP, ao banco e a um `ViewModel`? Por quê?
8. Quando vale a pena criar a camada de domínio com casos de uso?

Exercícios

Exercício 1 — Diagnóstico

Analise um projeto antigo seu (ou um exemplo da internet) e liste as responsabilidades misturadas. Desenhe como ficaria em camadas.

Exercício 2 — UiState

Modele o `UiState` da tela de reserva: bancada escolhida, data, horário inicial e final, validação de conflito, envio em andamento e mensagem de erro.

Exercício 3 — Fake

Crie `FakeBancadaRepository` que guarda os dados em um `MutableStateFlow`. Use-o para alimentar o `@Preview` e, no Módulo 14, os testes do ViewModel.

LabMonitor — Etapa 8

Refatore o app para MVVM com Hilt: `PainelViewModel`, `EquipamentosViewModel`, repositórios com implementação *fake* e o caso de uso `VerificarAlertasUseCase`.

Para aprofundar

- Guia de arquitetura de apps: `https://developer.android.com/topic/architecture`
- Recomendações de arquitetura: `https://developer.android.com/topic/architecture/recommendations`
- Camada de UI: `https://developer.android.com/topic/architecture/ui-layer`
- Camada de dados: `https://developer.android.com/topic/architecture/data-layer`
- Hilt: `https://developer.android.com/training/dependency-injection/hilt-android`
- Now in Android (app de referência): `https://github.com/android/nowinandroid`

Respostas das questões de revisão

1. A rotação refaz requisições e perde dados; as regras não podem ser testadas sem o app; uma mudança na API ou no banco quebra a tela; o código fica enorme e difícil de entender.
2. No MVP, o *presenter* chama métodos de uma interface da *view*. No MVVM, o `ViewModel` expõe estado e a *view* o observa, sem que o `ViewModel` a conheça.
3. Um único objeto imutável evita estados contraditórios e atualizações pela metade; cada mudança gera uma cópia nova, fácil de comparar, registrar e testar.
4. As camadas internas não conhecem as externas. Violação: um caso de uso que importa `android.content.Context` ou classes do Retrofit.
5. O `ViewModel` depende só da interface, e o Hilt injeta `FonteBle` ou `FonteMqtt`: trocar é mudar uma linha no módulo de DI. Nos testes, entra uma `FonteFake`.
6. Na DI, a classe recebe as dependências de fora, explícitas no construtor. No localizador, ela mesma as pede a um registro global, o que as esconde e só falha em execução.
7. Cliente HTTP e banco: únicos no app (`@Singleton`), pois são caros e compartilhados. `ViewModel`: ligado à tela, vive enquanto ela estiver na pilha.
8. Quando regras se repetem em mais de um `ViewModel`, são complexas ou precisam de testes isolados. Em apps pequenos, UI e dados bastam.

Fim do Módulo 08

Próximo: Módulo 09 (comum) — Fundamentos de banco de dados: modelagem, SQL, normalização, transações e índices.